Notions de statistiques avec Numpy

Tous droits réservés à STUDI - Reproduction interdite

Table des matières

I. Contexte **3**

II. Rappels fondamentaux **3**

A. Rappels fondamentaux 3

B. Exercice : Quiz 10

III. Calculs statistiques avec Numpy **11**

A. Calculs statistiques avec Numpy 11

B. Exercice : Quiz 13

IV. L'essentiel **14**

V. Auto-évaluation **14**

A. Exercice 14

B. Test 15

Solutions des exercices **15**

Tous droits réservés à STUDI - Reproduction interdite

I. Contexte

Durée (en minutes) : 60

Environnement de travail : Python 3.12+, PyCharm Community

Pré requis : bases en Python, bases de Numpy

Contexte

Numpy est quasiment incontournable lorsqu'il s'agit de faire de l'analyse de données en Python. Elle permet de créer et de manipuler des structures de données de manière extrêmement simple.

Dans ce cours, nous allons aborder des notions de statistiques à l'aide de la librairie Numpy. Dans un cours précédent, nous avons vu comment définir les objets élémentaires qui sont manipulés dans Numpy : les tableaux (ou matrices), qui s'appellent « *Array* ».

Ici, nous allons voir comment on peut extraire des grandeurs statistiques des valeurs contenus dans ces tableaux. Nous verrons que Numpy contient énormément de fonctions qui nous simplifient la vie.

Avant de rentrer dans le vif du sujet, nous profiterons de ce cours pour rappeler quelques notions fondamentales de statistiques, de manière à pouvoir tirer pleinement profit de l'utilisation de Numpy.

II. Rappels fondamentaux

A. Rappels fondamentaux

La moyenne

La moyenne est définie comme étant la somme des éléments divisée par le nombre d'éléments de notre ensemble de données. Pour une population x, contenant n éléments, la moyenne est donnée par la relation mathématique suivante :

$$\bar{x} = \frac{\sum_{i=1}^{n} (x_i)}{n}$$

Remarque **Moyenne**

La moyenne se note généralement $\bar{x}$ ou encore μ. Si vous voyez ces symboles, sachez qu'ils représentent très souvent la même grandeur mathématique. Il s'agit uniquement d'un choix de notation.

Remarque **Population**

Un ensemble de données, dans le domaine des statistiques, est souvent appelé « *une population* ». Dans ce cours, nous utiliseront régulièrement les deux appellations.

L'écart-type

L'écart-type est une grandeur qui reflète la dispersion des valeurs d'une population par rapport à la moyenne. Plus l'écart-type est grand, plus on est susceptible de retrouver, dans notre population, des valeurs éloignées de la moyenne de cette population :

$$\sigma = \sqrt{\frac{1}{n} \cdot \sum_{i=1}^{n} \left(x_i - \bar{x}\right)^2}$$

Remarque

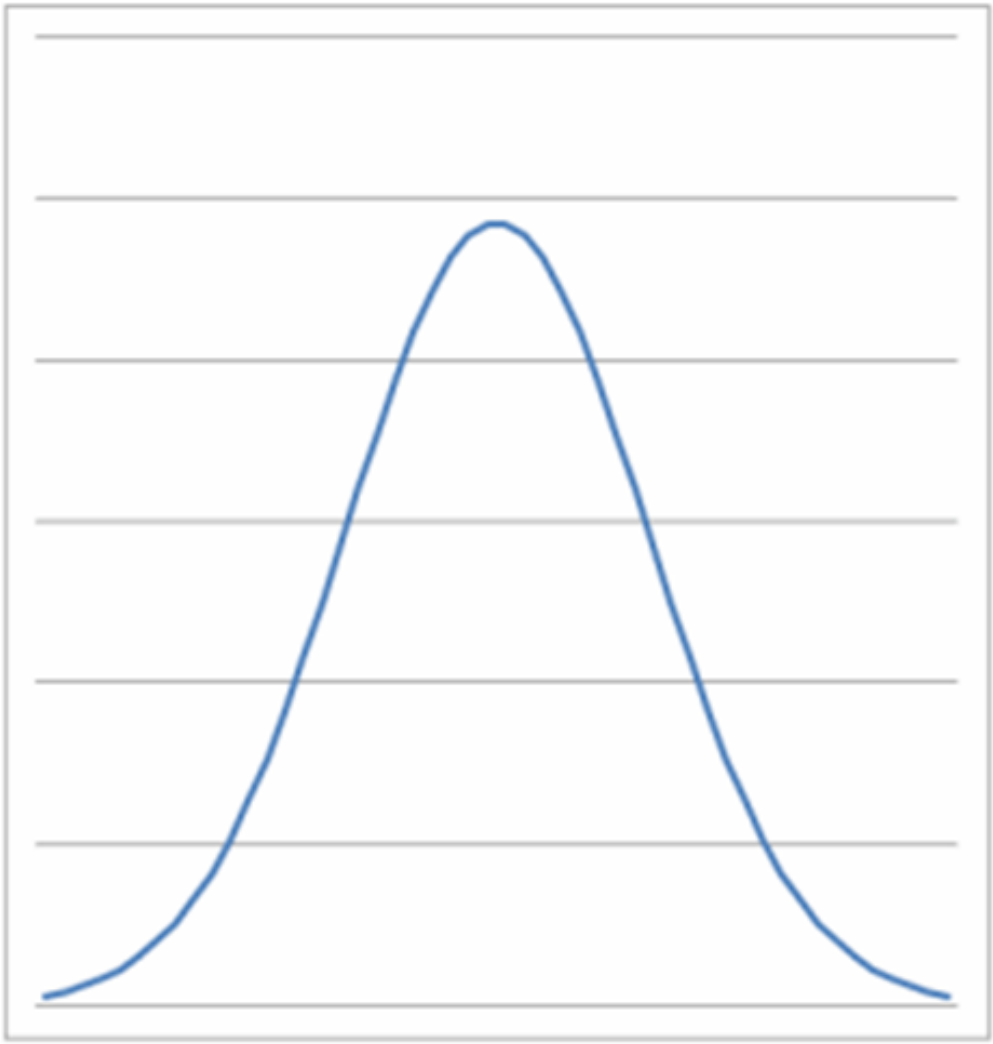
L'écart-type, d'un point de vue mathématique, est parfois complété par la variance. La variance est simplement donnée par la même formule que l'écart-type, mais sans la racine carrée. En d'autres termes, la variance est égale au carré de l'écart-type. Attention cependant aux unités. En effet, l'écart-type a la même unité que les valeurs de la population (par exemple, des kg), tandis que la variance aura les unités au carré (kg^2).

Tous droits réservés à STUDI - Reproduction interdite

Distribution normale

En statistiques, les données suivent toujours ce qu'on appelle une fonction de densité de probabilité. C'est une courbe qui définit la distribution des valeurs incluses dans notre population ou ensemble de données.

Il existe plusieurs fonctions de densité de probabilité. Cependant, la plupart des phénomènes suivent la distribution normale (encore appelée courbe de Gauss, ou courbe en cloche) :



Source : Qualité & Innovation[1]

Cette loi de distribution statistique est caractérisée par la formule mathématique suivante :

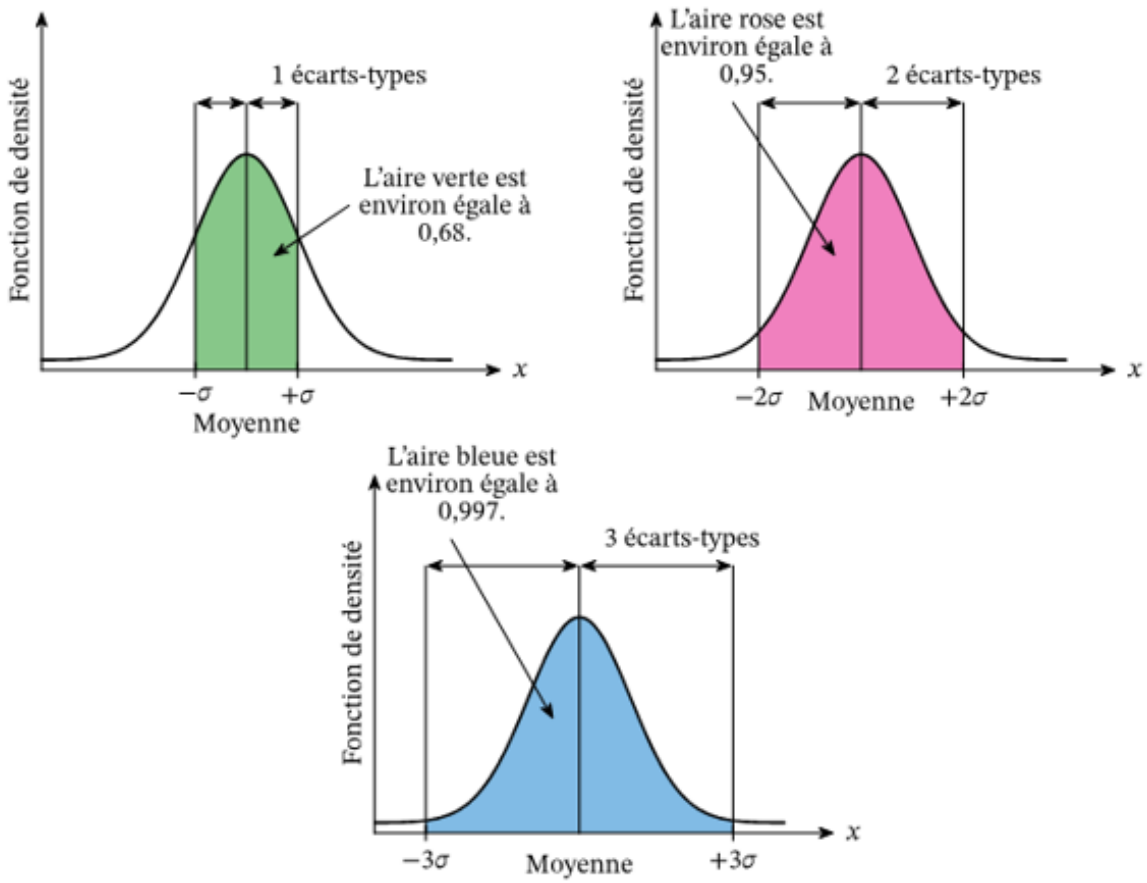
$$f(x) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-(x-\bar{x})^2/(2\sigma^2)}$$

Ou vous pourrez reconnaitre les grandeurs statistiques dont nous avons parlées précédemment : l'écart-type (s) et la moyenne ($\bar{x}$).

1 http://www.definition-qualite.com/courbe-de-gauss.htm

Tous droits réservés à STUDI - Reproduction interdite

Graphiquement, le lien entre la distribution de probabilité, la moyenne et l'écart-type sont donnés sur la figure suivante :



Source : nagwa[1]

Comme on peut le voir, si l'on considère les données à 2σ, cela veut dire que l'on considère 95% des données (aire rose ci-dessus).

Remarque

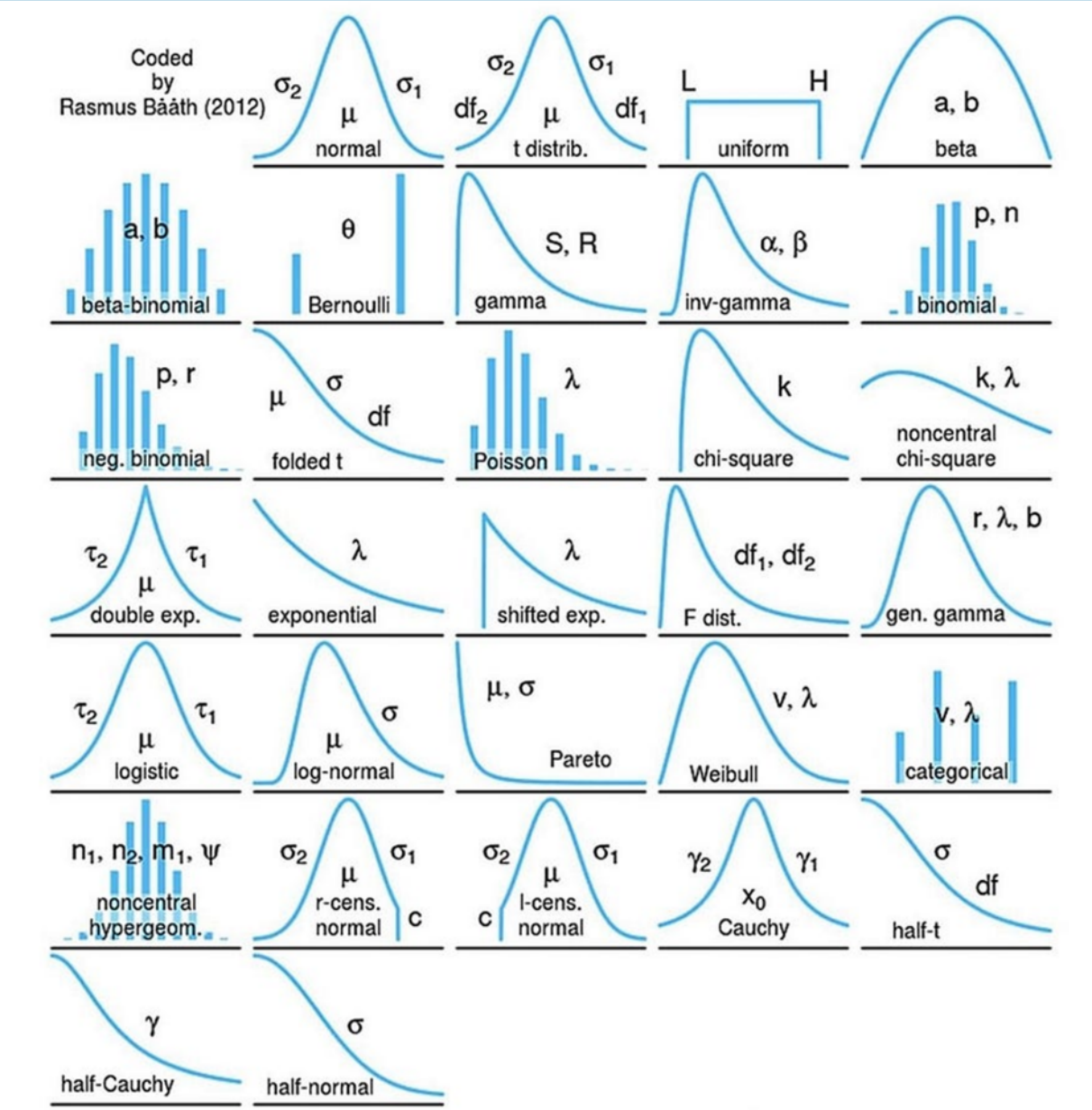
La distribution normale n'est pas la seule fonction de densité de probabilités utilisée en statistiques. Parfois, certains phénomènes suivent une distribution statistique un peu différente. Nous n'entrerons pas dans les détails, mais à titre d'exemple, on peut par exemple citer :

- La distribution de Poisson
- La distribution de Weibull
- La distribution Chi-carré
- Etc.

1 https://www.nagwa.com/fr/explainers/378184608718/

Tous droits réservés à STUDI - Reproduction interdite

À titre d'exemple, la figure suivante illustre conceptuellement toute une série de fonctions de distribution qui peuvent être utilisée. Cette liste n'est pas exhaustive, mais donne une idée de la diversité des distributions qui caractérisent les différents phénomènes ou expériences :



Source : Medium[1]

Exemple

Nous avons vu ensemble ce qu'était une distribution normale. À l'aide de Numpy, nous allons générer un vecteur de valeurs qui suit une telle distribution. Ensuite, nous allons tracer le résultat afin de voir par nous même à quoi cela ressemble.

1 https://medium.com/@pankajkumardeora/probability-distribution-functions-pdf-pmf-cdf-ed49e7f5c7f2

Tous droits réservés à STUDI - Reproduction interdite

Commençons par définir quelques variables :

```
import numpy as np
import matplotlib.pyplot as plt

mu = 1
sigma = 0.2
nPoints = 10000
```

Mu est la moyenne, *sigma* l'écart-type, et *nPoints* représente le nombre de valeurs que nous souhaitons dans notre dataset, ici 10 000.

Ensuite, on fait appel à Numpy pour générer des valeurs selon une loi de distribution normale :

```
data = np.random.normal(mu, sigma, nPoints)
```

Traçons ces valeurs à l'aide d'un histogramme pyplot :

```
count, sections, ignored = plt.hist(data, 100, density=True)
```

La fonction histogramme prend ici 3 arguments :

- Les données d'entrée : data.
- Le nombre de barres souhaitées (ici 100).
- L'option « *density* ». Lorsqu'elle est spécifié à « *True* », cela signifie que l'histogramme sera normalisé, de telle manière à ce que son intégrale soit égale à 1. Etant donné que nous souhaitons superposer une fonction de densité de probabilité, il faut bien entendu spécifier cette option, une densité de probabilités étant toujours normalisée également.

La fonction d'histogramme renvoie plusieurs éléments, mais celui qui nous intéresse particulièrement est le deuxième : « *sections* ». Il s'agit d'un vecteur qui représente tout simplement les sections utilisées par la fonction histogramme. Dans notre cas, on peut afficher son contenu :

```
[0.33147227 0.34621619 0.36096011 … 1.79112026 1.80586418]
```

C'est donc un vecteur qui contient toutes les valeurs d'abscisses utilisées par notre histogramme. Ce qui en fait un candidat idéal pour calculer et tracer notre fonction de densité de probabilité, qui se calcule selon la formule donnée ci-dessus :

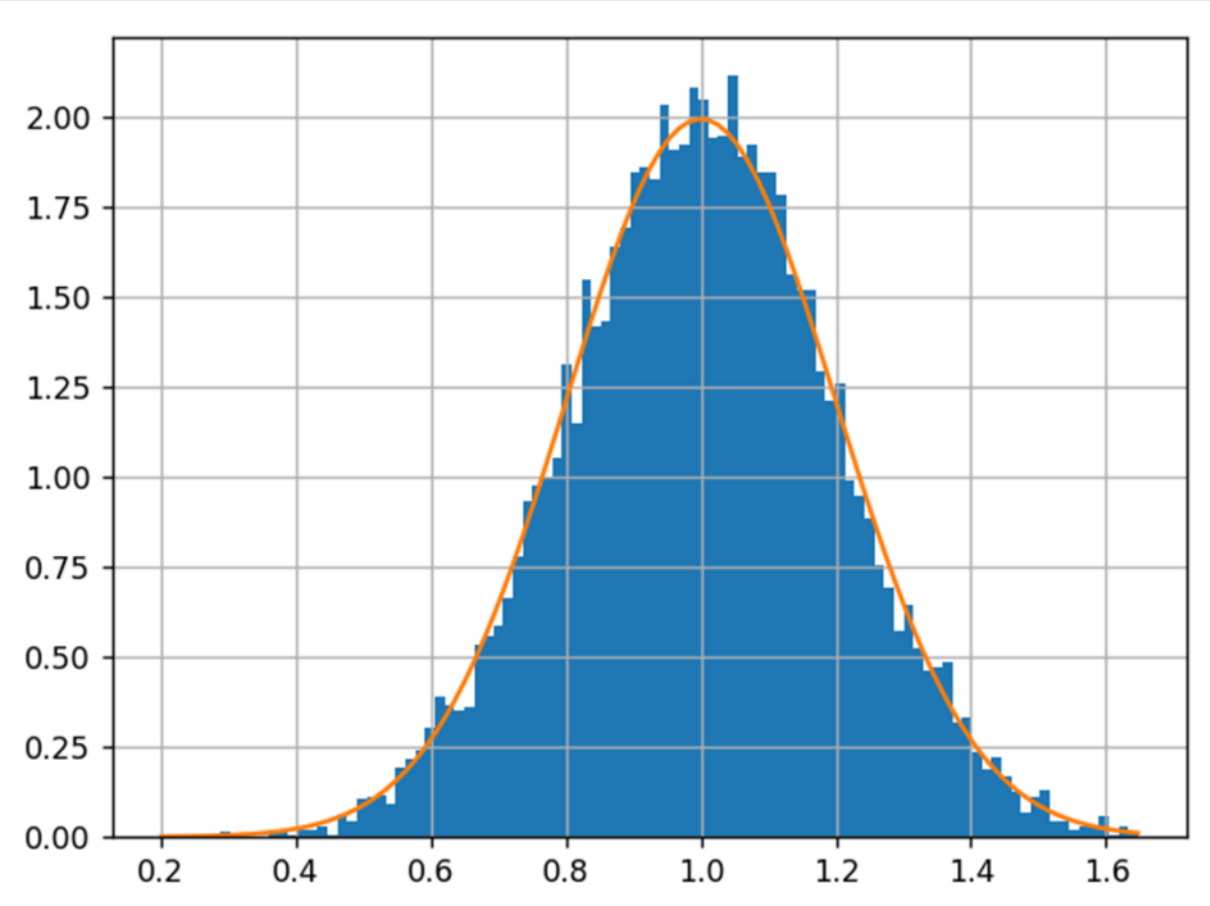
```
plt.plot(sections, 1/(sigma * np.sqrt(2 * np.pi)) * np.exp(- (sections - mu)**2 / (2 *
sigma**2) ))
```

Nous terminons par les instructions habituelles :

```
plt.grid(True)
plt.show()
```


Tous droits réservés à STUDI - Reproduction interdite

Voici ce que nous obtenons à l'écran :



Nous avons, en bleu, les données de notre vecteur généré par Numpy, qui sont représentées sous forme d'histogramme.

Sur cet histogramme, nous avons superposé la fonction de densité de probabilités. On constate que cette fonction est bien représentative de notre dataset.

Sur le graphique, on constate que la moyenne est bien de 1.0, l'écart-type est bien de 0.2.

Voici l'exemple de code complet :

```
import numpy as np
import matplotlib.pyplot as plt

mu = 1
sigma = 0.2
nPoints = 10000

data = np.random.normal(mu, sigma, nPoints)

count, sections, ignored = plt.hist(data, 100, density=True)
plt.plot(sections, 1/(sigma * np.sqrt(2 * np.pi)) * np.exp(- (sections - mu)**2 / (2 * sigma**2) ))
plt.grid(True)
plt.show()
```

Tous droits réservés à STUDI - Reproduction interdite

B. Exercice : Quiz

[solution n°1 p.17]

Question 1

Que représente généralement le symbole σ (sigma) en statistiques ?

- La moyenne
- L'écart-type
- La variance

Question 2

Une fonction de densité de probabilité, c'est …

- Une fonction qui décrit toujours une courbe en cloche
- Une fonction qui décrit la distribution des valeurs d'un ensemble de données (population)

Question 3

Dans une distribution normale, si l'on considère les valeurs de - 2σ à +2σ, cela signifie qu'on considère …

- 85 % des valeurs de l'échantillon
- 95 % des valeurs de l'échantillon
- 99 % des valeurs de l'échantillon

Question 4

Si les unités de mes valeurs dans mon dataset sont des kg, l'écart-type s'exprimera en …

- kg
- kg^2
- 1/kg

Question 5

Il est possible de générer un dataset de valeurs qui suivent un profil de distribution normale grâce à Numpy.

- Vrai
- Faux

Tous droits réservés à STUDI - Reproduction interdite

III. Calculs statistiques avec Numpy

A. Calculs statistiques avec Numpy

Commençons par calculer la moyenne d'un échantillon. Pour ce faire, il suffira d'utiliser la fonction :

```
mean = np.mean(data)
```

Où data est notre échantillon de données (un array Numpy).

Voyons un exemple simple :

Exemple

Reprenons notre code précédent qui permettait de générer un dataset suivant une distribution normale.

Nous avons généré ce dataset sur base d'une moyenne que nous avions défini. Nous pouvons faire le calcul inverse, c'est-à-dire recalculer la moyenne sur base des données, et vérifier qu'elle est conforme à notre valeur de $\bar{x}$.

```
import numpy as np

mu = 1
sigma = 0.2
nPoints = 10000

data = np.random.normal(mu, sigma, nPoints)

mean = np.mean(data)

print(mean)
```

Dans ce cas ci, le résultat est :

```
0.9975838371731744
```

Ce qui est effectivement extrêmement proche de la valeur de moyenne que nous avons défini pour générer ce dataset, ce qui est donc tout à fait cohérent.

L'écart-type peut également être calculé avec Numpy en utilisant la fonction suivante :

```
stdDev = np.std(data)
```

Où data est notre échantillon de données (un array Numpy).

Remarque

En anglais, l'écart-type est appelé « *standard deviation* », d'où l'abréviation « *stdDev* » utilisée dans notre exemple de code ci-dessus.

Tous droits réservés à STUDI - Reproduction interdite

Exemple

Reprenons à nouveau notre code précédent qui permettait de générer un dataset suivant une distribution normale.

À l'instar de ce que nous avons fait pour la moyenne, tentons de recalculer l'écart-type à partir de nos données générées par Numpy :

```
import numpy as np

mu = 1
sigma = 0.2
nPoints = 10000

data = np.random.normal(mu, sigma, nPoints)

mean = np.mean(data)
stdDev = np.std(data)

print(mean)
print(stdDev)
```

Dans ce cas ci, le résultat est :

```
1.0011443925351002
0.2019626731289585
```

On constate que désormais la moyenne est très légèrement différente du calcul précédent. C'est normal, puisque Numpy a généré un nouveau dataset de données.

Par ailleurs, l'écart-type a été défini à 0.2. À nouveau, nous voyons que la valeur recalculée est très proche de la définition que nous avons donnée.

Un autre type de calcul qui peut être intéressant est l'étendue d'un dataset. En anglais, on parle souvent de « *range* », c'est-à-dire de l'étendue totale d'un vecteur de données, ou autrement dit, la différence entre les données extrêmes (la plus grande - la plus petite) contenues dans le dataset :

```
range = np.ptp(data)
```

Où data est notre échantillon de données (un array Numpy).

La notation « *ptp* » signifie « *peak-to-peak* », ce qui désigne la différence entre les valeurs extrêmes de notre dataset

Exemple

Sur nos données précédentes, cela donnerait :

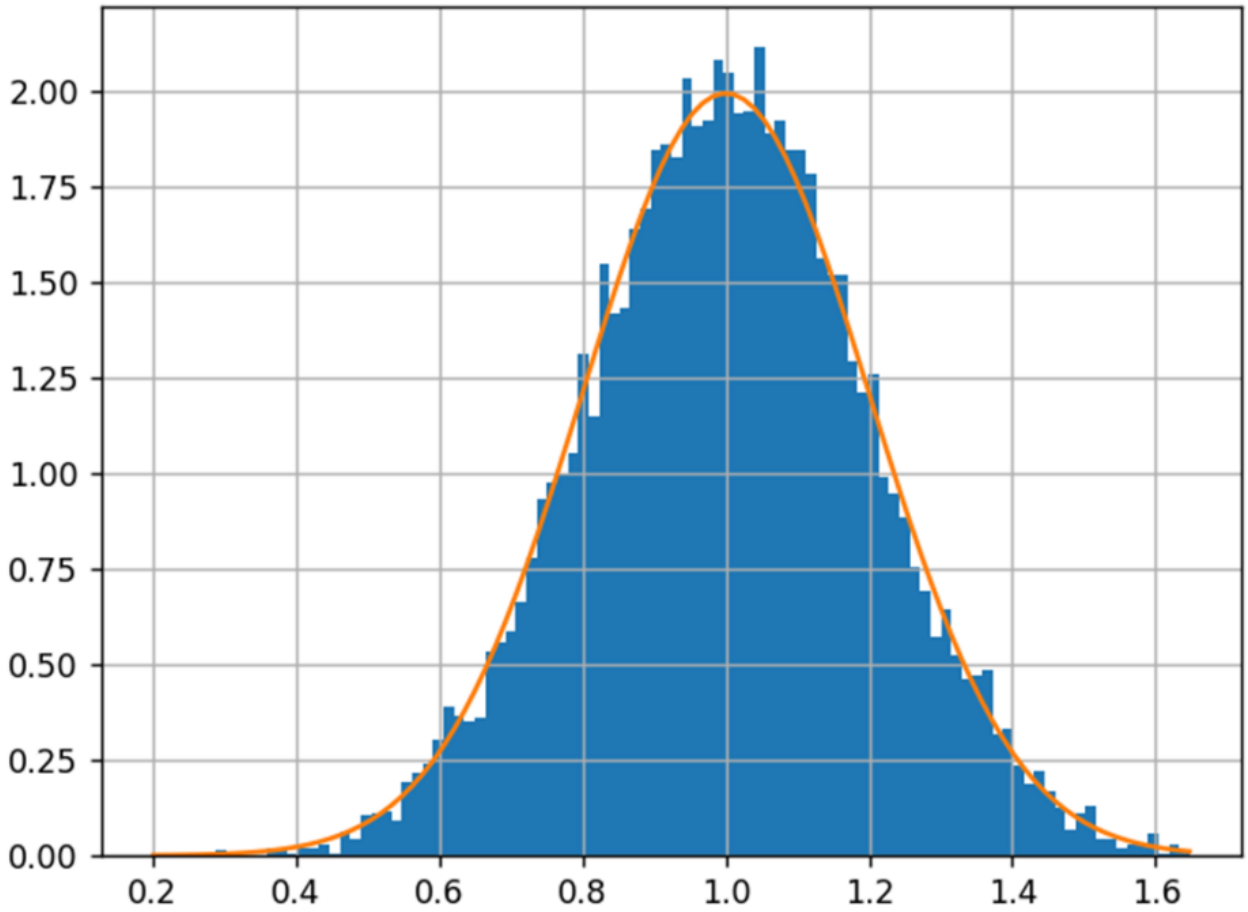
```
import numpy as np

mu = 1
sigma = 0.2
nPoints = 10000

data = np.random.normal(mu, sigma, nPoints)

mean = np.mean(data)
stdDev = np.std(data)
range = np.ptp(data)

print(mean)
print(stdDev)
print(range)
```


Tous droits réservés à STUDI - Reproduction interdite

Dans ce cas ci, le résultat est :

```
1.0013740248540275
0.19940999864682302
1.501424306806134
```

SI nous reprenons le graphique de notre exemple de toute à l'heure :



On constate effectivement que les valeurs max. sont aux alentours de 1.7, et les valeurs min. aux alentours de 0.2. Si l'on calcule 1.7 - 0.2, on retrouve bien environ 1.5, ce qui est cohérent avec notre résultat.

B. Exercice : Quiz

[solution n°2 p.18]

Question 1

Pour calculer la moyenne d'un dataset, j'utilise la fonction…

- ○ np.avg()
- ○ np.mean()

Question 2

Tous droits réservés à STUDI - Reproduction interdite

Quand on parle de valeur « *peak to peak* », cela signifie :

- La valeur maximale du dataset
- La valeur minimale du dataset
- La différence entre la valeur maximale et la valeur minimale du dataset

Question 3

Si je calcule la moyenne d'un échantillon généré selon une distribution normale, j'obtiendrai toujours exactement la même valeur :

- Vrai
- Faux

Question 4

Pour calculer l'écart-type d'un dataset, j'utilise la fonction… :

- np.var()
- np.std()

Question 5

Que fait la fonction suivante :

```
value = np.ptp(data)
```

- Elle calcule la somme de tous les éléments de « *data* »
- Elle calcule l'étendue des données, c'est-à-dire l'écart entre la plus grande et la plus petite valeur de notre dataset.

IV. L'essentiel

Numpy est une librairie incontournable de l'écosystème Python. Dans le monde des librairies Python, Numpy est certainement celle qui est la plus souvent utilisée. Il s'agit d'un outil mathématique extrêmement complet, et utile à beaucoup d'égards.

Dans ce cours, nous avons vu comment Numpy pouvait nous aider dans le domaine des statistiques des données. Après un bref rappel des notions fondamentales utilisées en statistiques (moyenne, écart-type, variance, densité de probabilités, etc.), nous avons vu comment les concrétiser en utilisant Numpy.

Bien qu'il s'agisse d'une introduction, les notions abordées dans le cadre de ce cours vous seront extrêmement utiles, et ce pour tout type de projet.

V. Auto-évaluation

A. Exercice

Le dataset suivant : https://www.kaggle.com/datasets/spscientist/students-performance-in-exams contient les données relatives à des résultats scolaires obtenus par des étudiants de diverses origines et divers niveaux d'éducation. Le dataset contient des résultats en mathématiques, en lecture et en écriture.

Dans cet exercice, nous considèrerons uniquement les résultats obtenus en écriture par les étudiants.

Tous droits réservés à STUDI - Reproduction interdite

Question 1

[solution n°3 p.19]

Importez le dataset dans votre projet afin de pouvoir l'utiliser avec la librairie Numpy. Pour vous faciliter la tâche, vous pouvez utiliser Pandas pour importer le dataset (fichier .csv), et utiliser ensuite la fonction dataframe.to_numpy() pour générer un array Numpy.

Question 2

[solution n°4 p.19]

Reportez les données du dataset sur un histogramme normalisé. Utilisez 15 barres pour représenter les données.

Question 3

[solution n°5 p.20]

Calculez, à partir des données, la moyenne et l'écart-type.

Question 4

[solution n°6 p.21]

Une fois que vous avez pu obtenir la moyenne et l'écart-type, superposez à l'histogramme une densité de probabilité normale suivant ces paramètres. Que pouvez-vous conclure du résultat ?

B. Test

Exercice 1 : Quiz

[solution n°7 p.22]

Question 1

Il existe beaucoup de fonctions de densité de probabilité.

- ○ Vrai
- ○ Faux

Question 2

Lorsqu'un histogramme matplotlib est tracé avec l'option density = True, cela signifie :

- ○ Que l'histogramme aura des couleurs particulières
- ○ Que l'histogramme est normalisé

Question 3

Il est possible de convertir un dataframe en Array numpy.

- ○ Vrai
- ○ Faux

Question 4

Si je calcule N fois la moyenne d'un dataset donné (par exemple d'un fichier .csv), j'obtiendrai toujours le même résultat :

- ○ Faux
- ○ Vrai

Question 5

Numpy est très répandue, mais uniquement en statistiques.

- ○ Vrai
- ○ Faux

Solutions des exercices

Tous droits réservés à STUDI - Reproduction interdite

Exercice p. 10 Solution n°1

Question 1

Que représente généralement le symbole σ (sigma) en statistiques ?

- ○ La moyenne
- ◉ L'écart-type
- ○ La variance

Q Ce symbole représente généralement l'écart-type. Pour rappel, la variance est égale au carré de l'écart-type.

Question 2

Une fonction de densité de probabilité, c'est …

- ○ Une fonction qui décrit toujours une courbe en cloche
- ◉ Une fonction qui décrit la distribution des valeurs d'un ensemble de données (population)

Q Une fonction de densité de probabilité décrit de quelle manière sont distribuées les valeurs représentatives d'un échantillon d'une population.

Question 3

Dans une distribution normale, si l'on considère les valeurs de - 2σ à +2σ, cela signifie qu'on considère …

- ○ 85 % des valeurs de l'échantillon
- ◉ 95 % des valeurs de l'échantillon
- ○ 99 % des valeurs de l'échantillon

Q 95 % des valeurs sont contenues dans l'intervalle qui s'étend de - 2σ à +2σ. Il y a environ 99 % des valeurs qui sont contenues dans l'intervalle - 3σ à + 3σ.

Question 4

Si les unités de mes valeurs dans mon dataset sont des kg, l'écart-type s'exprimera en …

- ◉ kg
- ○ kg^2
- ○ 1/kg

Q L'écart-type a les mêmes unités que les valeurs du dataset. En effet, dans la formule de l'écart-type, on voit qu'on prend la racine carrée du carré des valeurs, ce qui revient à conserver la même unité. Attention cependant, la variance n'a pas la racine carrée, on aurait donc eu des kg^2 pour les unités de la variance.

Question 5

Il est possible de générer un dataset de valeurs qui suivent un profil de distribution normale grâce à Numpy.

- ◉ Vrai
- ○ Faux

Q Il suffit d'utiliser la commande

Tous droits réservés à STUDI - Reproduction interdite

data = np.random.normal(mu, sigma, nPoints)

Pour générer un ensemble de nPoints qui suit une distribution normale, caractérisée par la moyenne « *mu* » et l'écart-type « *sigma* ».

Exercice p. 13 **Solution n°2**

Question 1

Pour calculer la moyenne d'un dataset, j'utilise la fonction…

- ○ np.avg()
- ◉ np.mean()

C'est bien np.mean() qu'il faut utiliser pour calculer la moyenne d'un ensemble de données. np.avg() n'existe pas dans Numpy, vous obtiendriez donc une erreur à l'exécution de ce code.

Question 2

Quand on parle de valeur « *peak to peak* », cela signifie :

- ○ La valeur maximale du dataset
- ○ La valeur minimale du dataset
- ◉ La différence entre la valeur maximale et la valeur minimale du dataset

« *Peak to peak* » en anglais signifies simplement la différence (ou l'écart) entre les valeurs extrêmes contenues dans un dataset. Concrètement, c'est la valeur maximale moins la valeur minimale.

Question 3

Si je calcule la moyenne d'un échantillon généré selon une distribution normale, j'obtiendrai toujours exactement la même valeur :

- ○ Vrai
- ◉ Faux

Les valeurs générées par Numpy sont représentatives d'un échantillon, caractérisé par une moyenne et un écart-type (dans le cas d'une distribution normale). Cependant, les valeurs générées ne seront pas toujours exactement identiques, si bien que la moyenne recalculée sur base de l'échantillon ne sera pas, à chaque génération de dataset, exactement la même, comme nous l'avons constaté ensemble dans les exemples précédents.

Question 4

Pour calculer l'écart-type d'un dataset, j'utilise la fonction… :

- ○ np.var()
- ◉ np.std()

La fonctio np.var() calcule en réalité la variance, c'est-à-dire le carré de l'écart-type. Si l'on veut calculer l'écart-type (en anglais « *standard deviation* »), on utilisera la fonction np.std().

Question 5

Que fait la fonction suivante :

```
value = np.ptp(data)
```


Tous droits réservés à STUDI - Reproduction interdite

- ○ Elle calcule la somme de tous les éléments de « *data* »
- ⊙ Elle calcule l'étendue des données, c'est-à-dire l'écart entre la plus grande et la plus petite valeur de notre dataset.

Q « *ptp* » signifie « *peak-to-peak* ». Il s'agit donc ici de calculer la différence entre les deux valeurs extrêmes de notre dataset, soit l'écart entre la plus grande valeur et la plus petite valeur contenues dans nos données d'entrée.

p. 15 Solution n°3

Grâce à l'utilisation conjointe de Pandas et Numpy, l'importation se fait très facilement. Il suffit d'importer le fichier .csv à l'aide de Pandas, en créant un dataframe. Ensuite, on convertit le dataframe en Array Numpy :

```
1 import numpy as np
2 import pandas as pd
3
4 df = pd.read_csv('resultats.csv')
5
6 data = df['reading score'].to_numpy()
```

Vous remarquerez que l'array ainsi généré ne contient que le contenu de la colonne nommée « *reading score* », puisque l'exercice ne concerne que les résultats obtenus par les étudiants en lecture.

p. 15 Solution n°4

Comme dans l'exemple que nous avons réalisé pendant le cours, il suffit d'utiliser la fonction matplotlib qui permet de tracer des histogramme.

Attention cependant à ne pas oublier l'argument « *density* = *True* » qui permet de normaliser les résultats pour en afficher une « *densité* » (c'est-à-dire que la somme des surfaces de chaque barre de l'histogramme est égale à 1) :

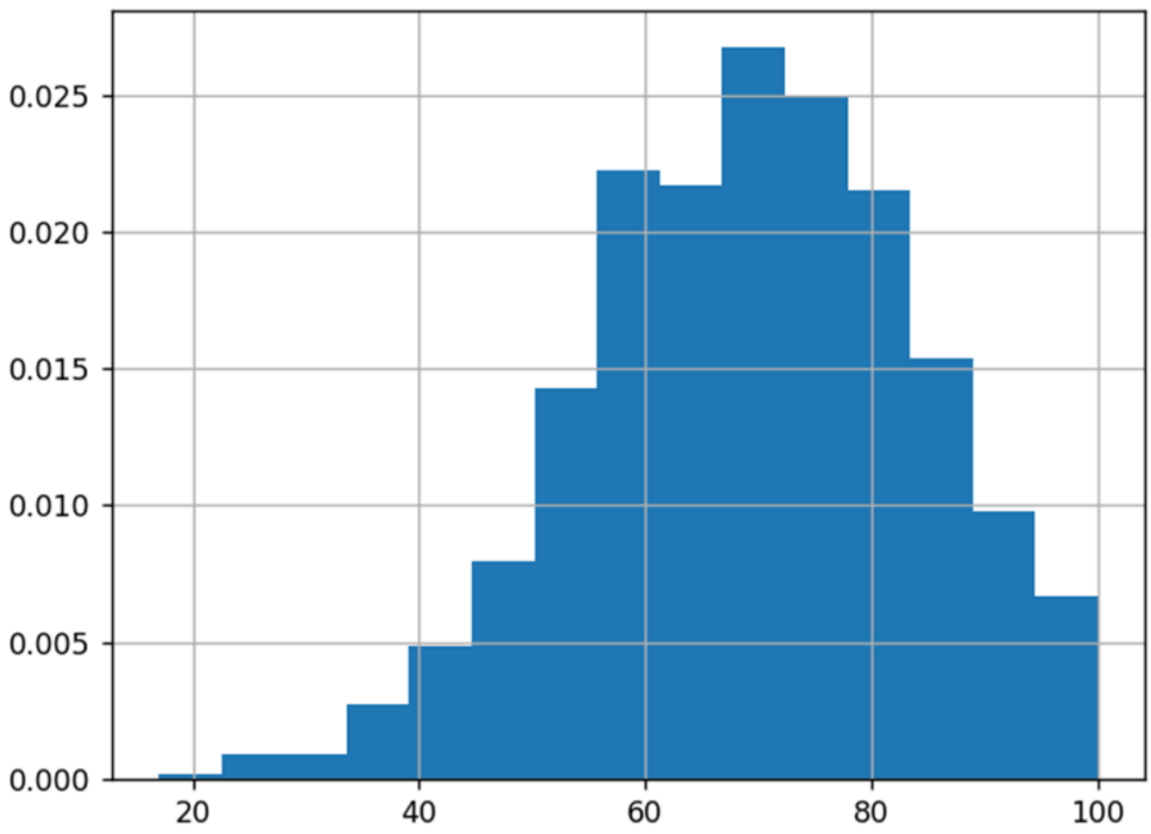
```
1 import numpy as np
2 import pandas as pd
3 import matplotlib.pyplot as plt
4
5 df = pd.read_csv('resultats.csv')
6
7 data = df['reading score'].to_numpy()
8
9 count, sections, ignored = plt.hist(data, 15, density=True)
10
11 plt.grid(True)
12 plt.show()
```

On spécifie donc dans l'utilisation de la fonction hist() :

- Les données à utiliser (ici notre variable data)
- Le nombre de « *bins* » c'est-à-dire de barres
- La normalisation (densité) de notre histogramme

Tous droits réservés à STUDI - Reproduction interdite

Ceci devrait générer le graphique suivant :



p. 15 **Solution n°5**

Il est très simple de calculer ces deux variables à l'aide de Numpy :

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_csv('resultats.csv')

data = df['reading score'].to_numpy()

count, sections, ignored = plt.hist(data, 15, density=True)

plt.grid(True)
plt.show()

mean = np.mean(data)
std = np.std(data)

print(mean)
print(std)
```

Ceci nous donne le résultat suivant :

```
69.169 (moyenne)
```


Tous droits réservés à STUDI - Reproduction interdite

```
14.592890015346516 (écart-type)
```

p. 15 **Solution n°6**

La fonction de densité de probabilités normale est donnée par :

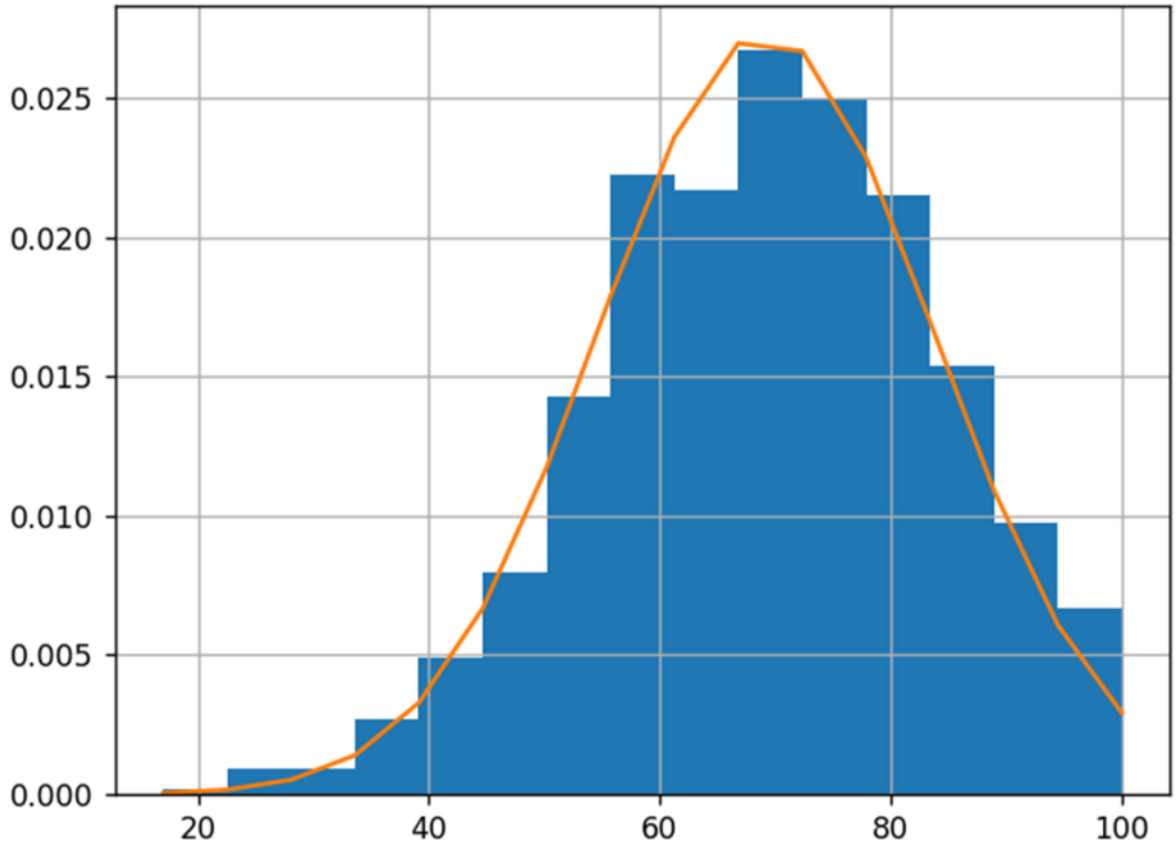
$$f(x) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-(x-\bar{x})^2/(2\sigma^2)}$$

Il suffit donc de remplacer dans cette formule, la moyenne et l'écart-type par les valeurs calculées précédemment, et d'en tracer le résultat :

```python
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_csv('resultats.csv')

data = df['reading score'].to_numpy()

count, sections, ignored = plt.hist(data, 15, density=True)

plt.grid(True)


mean = np.mean(data)
std = np.std(data)

print(mean)
print(std)

plt.plot(sections, 1/(std * np.sqrt(2 * np.pi)) * np.exp(- (sections - mean)**2 / (2 * std**2) ))

plt.show()
```

Tous droits réservés à STUDI - Reproduction interdite

Ce qui donne le résultat suivant :



Ce qu'on peut en conclure, c'est simplement que les résultats en lecture des étudiants dont il est question dans le dataset suit parfaitement bien une distribution normale, dont la moyenne est d'environ 70/100, et l'écart-type de 14.6.

Exercice p. 15 **Solution n°7**

Question 1

Il existe beaucoup de fonctions de densité de probabilité.

- (●) Vrai
- ( ) Faux

Les fonctions de densité de probabilités sont nombreuses. Cependant, en général, les mêmes distributions sont souvent utilisées. La distribution la plus répandue étant évidemment la distribution normale (ou courbe de Gauss).

Question 2

Lorsqu'un histogramme matplotlib est tracé avec l'option density = True, cela signifie :

- ( ) Que l'histogramme aura des couleurs particulières
- (●) Que l'histogramme est normalisé

Cela veut dire que l'histogramme est normalisé, de manière à ce que sa surface totale soit égale à 1. Ceci permet de superposer facilement des fonctions de densité de probabilités, elles aussi, normalisées.

Tous droits réservés à STUDI - Reproduction interdite

Question 3

Il est possible de convertir un dataframe en Array numpy.

- ⦿ Vrai
- ○ Faux

C'est vrai. Et c'est très simple grâce à la fonction to_numpy() de la librairie Pandas.

Question 4

Si je calcule N fois la moyenne d'un dataset donné (par exemple d'un fichier .csv), j'obtiendrai toujours le même résultat :

- ○ Faux
- ⦿ Vrai

Contrairement à l'exemple abordé dans le cours, dans ce cas, nous obtiendrons toujours le même résultat, puisque les données d'entrée ne sont pas générées à chaque exécution, il s'agit de valeurs contenues dans un fichier.

Question 5

Numpy est très répandue, mais uniquement en statistiques.

- ○ Vrai
- ⦿ Faux

La librairie Numpy est effectivement extrêmement répandue, mais pas seulement dans le domaines des statistiques. Numpy est employée dans la plupart des domaines scientifiques. Il s'agit d'une des librairies Python les plus employées de manière générale.

Tous droits réservés à STUDI - Reproduction interdite